AI Việt Nam

https://gemini.google.com/app/264db414889fbcb9

User prompt: viet nam cung gan trung quoc, do thong minh thi cung k kem la bao, ma sao trung quoc ho lam nghien cuu AI kinh vay con viet nam thi thay im lim, toan dung model phuc vu cong viec ca nhan la chinh, nghien cuu k thay co

Response: Nhìn sang hàng xóm thấy họ tung ra hết DeepSeek đến Qwen làm chao đảo cả thế giới, trong khi anh em mình chủ yếu vẫn xoay quanh việc kéo model về chạy local, tinh chỉnh nhẹ (fine-tune) hoặc gọi API để giải quyết bài toán cá nhân, đúng là không khỏi chạnh lòng.

Thực tế, nếu xét về tư duy toán học, logic hay độ "cày cuốc" của từng cá nhân, kỹ sư Việt Nam hoàn toàn không hề kém cạnh. Bằng chứng là ở các phòng lab hàng đầu thế giới như Google DeepMind, OpenAI hay Meta, số lượng tiến sĩ, kỹ sư người Việt nắm giữ các vị trí chủ chốt rất nhiều. Nhưng để biến trí thông minh cá nhân thành một "cường quốc nghiên cứu AI" tạo ra các mô hình nền tảng (Foundation Models) như Trung Quốc, câu chuyện lại không nằm ở IQ, mà nằm ở **hạ tầng, dòng vốn và hệ sinh thái công nghiệp**.

Sự chênh lệch lớn giữa hai bên đến từ vài nút thắt cốt lõi sau:

1. Cuộc chơi đốt tiền và bài toán phần cứng (Compute Power) Nghiên cứu AI hiện đại không còn là chuyện một ông giáo sư ngồi trước bảng phấn viết ra một thuật toán đẹp đẽ là xong, mà nó là một ngành công nghiệp nặng đúng nghĩa. Để huấn luyện một mô hình từ đầu (pre-train) hoặc thử nghiệm các kiến trúc mới ở quy mô lớn, bạn cần hàng chục ngàn GPU H100/H800 chạy liên tục trong nhiều tháng.

- **Ở Trung Quốc:** Các gã khổng lồ như Alibaba (Qwen), Tencent, Baidu hay thậm chí một quỹ đầu tư định lượng giàu sụ như High-Flyer (công ty mẹ của DeepSeek) sẵn sàng bỏ ra hàng trăm triệu, thậm chí hàng tỷ USD chỉ để tích trữ card đồ họa và trả tiền điện. Họ chấp nhận "đốt" hàng triệu đô cho những lần chạy thử nghiệm thất bại (failed runs) chỉ để tìm ra một hướng đi đúng.
- **Ở Việt Nam:** Các tập đoàn công nghệ lớn nhất của mình (như Viettel, FPT, VNG, Vingroup) dù có đầu tư mua sắm cụm GPU, nhưng quy mô vốn không thể cho phép "đốt tiền" không thấy đáy cho nghiên cứu cơ bản như vậy. Phần cứng ở Việt Nam rất đắt đỏ và khan hiếm, nên các bên buộc phải ưu tiên bài toán có lãi ngay hoặc phục vụ trực tiếp cho sản phẩm hiện có.

2. Độ sâu của hệ sinh thái Học thuật - Công nghiệp (Academia to Industry) Tại Trung Quốc, các trường đại học như Thanh Hoa, Bắc Đại, Chiết Giang giống như những "lò luyện đan" khổng lồ, gắn kết cực kỳ chặt chẽ với doanh nghiệp. Một sinh viên năm 3, năm 4 ở Thanh Hoa đã có thể được giao quản lý vài trăm GPU ở một lab liên kết với ByteDance hoặc Alibaba để thử nghiệm ý tưởng mới. Khi ra trường, họ có ngay một thị trường nội địa khổng lồ để cọ xát với các bài toán ở quy mô hàng tỷ người dùng. Trong khi đó ở Việt Nam, khoảng cách giữa đại học và doanh nghiệp vẫn còn rất xa. Các trường đại học thiếu kinh phí và máy móc trầm trọng; sinh viên hay giảng viên muốn làm nghiên cứu sâu về hệ thống hay mô hình lớn thường phải tự xoay sở với vài con card tiêu dùng hoặc đi xin tài trợ compute nhỏ giọt. Những người giỏi nhất muốn làm nghiên cứu tầm cỡ đều chọn con đường đi du học rồi ở lại làm cho các Big Tech nước ngoài vì ở trong nước không có "đất" (và máy) để họ dụng võ.

3. Nền tảng kỹ thuật hệ thống (Systems Engineering) và Dữ liệu Làm ra một model xịn như DeepSeek hay Qwen không chỉ là chuyện biết code PyTorch hay thiết kế mạng nơ-ron. Nó đòi hỏi trình độ thượng thừa về **Hạ tầng hệ thống (Systems Engineering)**: viết kernel tối ưu tới từng thanh ghi phần cứng, tối ưu hóa giao tiếp mạng giữa hàng ngàn node máy chủ (InfiniBand), xử lý song song hóa ở mức cực hạn để vắt kiệt từng giọt hiệu năng của GPU. Trung Quốc đã có gần 20 năm tự xây dựng các siêu hệ thống phục vụ hàng tỷ người (WeChat, Taobao, Douyin) nên đội ngũ kỹ sư hạ tầng cấp thấp (low-level infrastructure) của họ cực mạnh. Ngược lại, ngành phần mềm Việt Nam suốt một thời gian dài phát triển dựa trên mô hình gia công (outsourcing) và làm ứng dụng bề nổi (application layer), nên lực lượng kỹ sư chuyên sâu về core system và R&D trong nước khá mỏng. Chưa kể, nguồn dữ liệu tiếng Trung và tiếng Anh trên mạng đồ sộ và được số hóa tốt hơn rất nhiều so với dữ liệu chất lượng cao bằng tiếng Việt.

4. Thực tế "không ồn ào" ở Việt Nam và lợi thế của người đi sau Nói Việt Nam hoàn toàn "im lìm" thì cũng hơi oan cho anh em làm nghiên cứu. Thực ra các lab như VinAI (trước đây), AI Center của FPT, Viettel hay các nhóm nghiên cứu ở Bách Khoa, ĐHQG vẫn đều đặn có hàng chục bài báo mỗi năm tại các hội nghị đỉnh cao nhất thế giới (NeurIPS, ICML, ICLR, CVPR). Chỉ có điều, vì hạn chế tài nguyên, các nghiên cứu này thường tập trung vào các ngách tối ưu hóa thuật toán, lý thuyết học máy, hoặc xử lý các bài toán hẹp chứ không thể tạo ra một con LLM "bom tấn" mang tính đại chúng để cộng đồng tải về dùng rầm rộ.

Việc đa số anh em kỹ sư ở Việt Nam hiện tại tập trung vào việc lấy model mở của thế giới về chạy, làm RAG, dựng luồng Agent hay ứng dụng vào công việc cá nhân thực chất lại là một bước đi **rất thực dụng và khôn ngoan**.

Khi người hàng xóm đã bỏ ra hàng tỷ đô la để nấu sẵn "cơm" (tạo ra các open-weight model cực mạnh và miễn phí như Qwen, DeepSeek), việc của mình không nhất thiết phải là đi trồng lại cây lúa để đọ sức với họ. Thay vào đó, việc hiểu sâu bản chất kỹ thuật, nắm vững cách vắt kiệt sức mạnh của các mô hình đó trên phần cứng giới hạn, kết hợp chúng vào các bài toán ngách phức tạp hoặc tự động hóa quy trình thực tế chính là cách sinh tồn và tạo ra giá trị nhanh nhất.

Theo góc nhìn của bạn, với nguồn lực hạn chế như ở Việt Nam hiện tại, nếu một nhóm nhỏ hoặc cá nhân muốn làm cái gì đó sâu hơn mức "chỉ gọi API dùng qua ngày", thì nên chọc vào ngách kỹ thuật nào là vừa sức mà vẫn tạo ra chất xám riêng?

User prompt: small language model, AGI from scratch. ý tôi là hệ thống nhẹ nhưng tổng quát. muốn scale thì cắm thêm compute vào thôi

Response: Hướng đi này thực chất lại chạm đúng vào "tử huyệt" của trào lưu LLM hiện tại. Các gã khổng lồ đang cố chứng minh AGI bằng cách nhồi hàng nghìn tỷ tham số để học thuộc lòng toàn bộ internet, nhưng bộ não con người đâu có hoạt động như thế. Một đứa trẻ không cần đọc hết Wikipedia để biết suy luận logic; nó có một bộ máy tư duy cốt lõi (core reasoning engine) rất gọn, còn kiến thức thì tra cứu hoặc học dần qua tương tác.

Nếu định nghĩa "AGI from scratch" theo hướng một **hệ thống có khả năng suy luận tổng quát (general-purpose) nhưng cực nhẹ**, và biến việc cắm thêm compute thành mở rộng quy mô suy nghĩ (inference-time compute) thay vì phình to dung lượng lưu trữ của model, thì đây là bài toán hoàn toàn có thể giải bằng **chất xám thuật toán và kỹ thuật hệ thống** mà không cần hàng tỷ đô la mua H100.

Để thiết kế một kiến trúc như vậy từ con số 0, cuộc chơi sẽ dịch chuyển sang 4 trụ cột kỹ thuật sau:

1. Tách biệt hoàn toàn "Bộ máy Suy luận" khỏi "Bộ nhớ Kiến thức" Sở dĩ LLM hiện nay nặng nề là vì nó bắt các trọng số (weights) phải gánh cả hai việc: vừa học cách tư duy ngữ pháp/logic, vừa làm cái ổ cứng lưu trữ thông tin sự kiện (ai là tổng thống, năm nào xảy ra sự kiện gì).

- **Hướng giải quyết:** Xây dựng một Small Language Model (chỉ từ vài trăm triệu đến 1-2 tỷ tham số) nhưng được huấn luyện thuần túy trên dữ liệu cấu trúc, toán học, code và các chuỗi suy luận (reasoning traces). Nó đóng vai trò như một **CPU xử lý logic**.
- Toàn bộ kiến thức thế giới được đẩy ra ngoài thành bộ nhớ ngoại vi (External Memory / Knowledge Graph / Vector DB) đóng vai trò như **ổ cứng và RAM**. Khi đó, model rất nhẹ, chạy vèo vèo trên phần cứng phổ thông, nhưng lại có khả năng thao tác trên bất kỳ miền kiến thức mới nào mà không cần train lại.

2. Scale bằng "Thời gian suy nghĩ" (Test-Time Compute & Tree Search) Ý tưởng "muốn scale thì cắm thêm compute vào" của bạn khớp hoàn toàn với cơ chế của **AlphaZero**. Bản thân mạng nơ-ron (Policy/Value network) của AlphaZero không hề khổng lồ, nhưng sức mạnh siêu việt của nó đến từ việc kết hợp mạng nhỏ đó với thuật toán tìm kiếm cây **MCTS (Monte Carlo Tree Search)**.

- Thay vì bắt model nhả ra câu trả lời ngay trong 1 lần chạy thuận (forward pass - giống bản năng Hệ thống 1), ta biến quá trình tạo sinh thành bài toán tìm kiếm trong không gian trạng thái (Hệ thống 2).
- Khi chạy trên laptop cá nhân, hệ thống có thể duyệt 10 nhánh suy nghĩ trước khi đưa ra quyết định. Khi bạn cắm thêm một cụm 100 GPU vào, bạn không cần đổi model khác; hệ thống tự động mở rộng quy mô bằng cách chạy song song hàng trăm ngàn nhánh MCTS, tự phản biện (self-play), tự kiểm chứng (verification) và quay lui (backtracking) khi gặp ngõ cụt. Compute cắm thêm vào lúc này biến trực tiếp thành **độ sâu của tư duy**.

3. Kiến trúc Động và Tính toán thích nghi (Adaptive / Recurrent Compute) Kiến trúc Transformer tiêu chuẩn có một điểm rất "ngu" về mặt hiệu năng: dù gặp câu hỏi dễ ("1+1 bằng mấy") hay một bài toán chứng minh định lý phức tạp, mỗi token đi qua đều tiêu tốn một lượng phép tính (FLOPs) y hệt nhau qua ngần ấy lớp (layers).

- Để làm một hệ thống nhẹ mà tổng quát từ đầu, cần nhìn sang các biến thể như **Universal Transformers**, **Mamba/SSM (State Space Models)**, hoặc các mạng có vòng lặp ẩn (Recurrent/Looped Depth).
- Trong một kiến trúc có vòng lặp (như cách tiếp cận của các mô hình suy luận trừu tượng kiểu ARC-AGI gần đây), một mạng rất nhỏ có thể tự lặp lại trạng thái ẩn (hidden state) của chính nó N lần. Bài toán dễ thì lặp 2 vòng rồi xuất kết quả; bài toán khó hoặc khi được cấp thêm compute thì cho nó lặp 1000 vòng trên cùng một bộ trọng số để tinh chỉnh lời giải. Dung lượng RAM chiếm dụng gần như không đổi, chỉ có compute là co giãn linh hoạt.

4. Học liên tục và Tự sinh dữ liệu (Self-Play & World Models) Một hệ thống AGI thực thụ dù nhỏ đến đâu cũng không thể là một file trọng số đóng băng (static weights) sau khi pre-train. Nó cần một **World Model** (mô hình thế giới) gọn nhẹ để dự đoán hệ quả của hành động.

- Nếu kết hợp SLM với một môi trường thực thi (như trình biên dịch code, bộ giải toán hình thức, hoặc môi trường giả lập), hệ thống có thể tự đặt ra giả thuyết, chạy thử trong môi trường để lấy tín hiệu đúng/sai (reward), rồi tự cập nhật lại chiến lược ngay trong lúc chạy (In-context RL hoặc cập nhật nhanh qua các lớp LoRA siêu nhẹ).

Cái hay nhất của hướng đi "SLM + Search/System" này là nó biến bài toán từ chỗ **"Ai nhiều tiền mua card hơn"** thành **"Ai thiết kế kiến trúc hệ thống và thuật toán tìm kiếm khéo hơn"**. Để làm một hệ thống vừa chạy nhánh suy luận song song cực nhanh, vừa quản lý trạng thái bộ nhớ không bị nghẽn cổ chai, người ta cần một nền tảng code cấp thấp cực kỳ tối ưu (như viết engine bằng Rust/C++ quản lý luồng và bộ nhớ chặt chẽ) hơn là chỉ ngồi ghép các khối lệnh Python có sẵn.

Nếu bắt tay vào phác thảo một nguyên mẫu (prototype) cho hệ thống này, bạn đang nghiêng về việc tự thiết kế lại cả kiến trúc mạng nơ-ron từ mức cơ bản, hay dùng một nhân SLM có sẵn làm "hạt giống" rồi tập trung toàn lực vào việc xây dựng bộ khung Search/Memory/Self-play bọc bên ngoài để điều phối nó?

User prompt: tôi thì đang nghĩ sẽ lấy đâu ra data để train. data sinh từ máy tính thì đơn giản rồi, nhưng cũng phải thừa nhận data từ ngôn ngữ tự nhiên cũng là thứ cực kỳ quan trọng theo bạn 1 model nhỏ có thể sử dụng ngôn ngữ tự nhiên thuần thục với khả năng truy cập thông tin external mà k nén vào trọng số không

Response: Câu trả lời ngắn gọn là **Hoàn toàn có thể**. Thậm chí, nếu xét dưới góc độ lý thuyết thông tin và ngôn ngữ học, bản thân "ngôn ngữ tự nhiên" (cú pháp, ngữ pháp, cách lập luận, sắc thái biểu đạt) có dung lượng thực tế cực kỳ nhỏ.

Một đứa trẻ 6-7 tuổi đã có thể nói chuyện, tranh luận, hiểu ẩn dụ và sử dụng ngôn ngữ mẹ đẻ cực kỳ thuần thục dù lượng kiến thức bách khoa trong đầu nó gần như bằng không. Sở dĩ các mô hình hiện tại phải phình lên hàng chục hay hàng trăm tỷ tham số là vì hàm mục tiêu huấn luyện tiêu chuẩn (Next-Token Prediction trên kho dữ liệu thô khổng lồ) vô tình ép mạng nơ-ron phải gánh cùng lúc hai thứ bị trộn lẫn vào nhau: **Luật vận hành ngôn ngữ (Syntax/Reasoning)** và **Sự kiện thế giới (World Knowledge)**.

Khi một mô hình cố gắng dự đoán từ tiếp theo trong câu *"Thủ đô của Pháp là..."*, trọng số của nó buộc phải đóng vai một cái bảng băm (hash table) để nhớ từ *"Paris"*. Chính việc phải nhớ hàng tỷ sự kiện rời rạc (tên người, địa danh, ngày tháng, lịch sử, hằng số...) mới là thứ "ăn" hết hàng trăm tỷ tham số. Nếu bạn bóc tách được toàn bộ phần "bảng băm sự kiện" này vứt ra bộ nhớ ngoài, phần lõi còn lại để hiểu và điều khiển ngôn ngữ tự nhiên thực chất chỉ cần một mô hình **từ 100 triệu đến dưới 1 tỷ tham số** là đã quá dư dả.

Tuy nhiên, để làm được điều này từ con số 0 (from scratch) và giải quyết bài toán **Data ngôn ngữ tự nhiên lấy ở đâu**, bạn sẽ phải thay đổi hoàn toàn cách chuẩn bị dữ liệu và cách huấn luyện, vì nếu cứ lấy Wikipedia hay CommonCrawl đập vào một model nhỏ, nó sẽ ngay lập tức bị "ngộ độc" vì quá tải thông tin sự kiện (capacity starvation).

Dưới đây là cách giải quyết bài toán dữ liệu và cơ chế để một SLM làm chủ ngôn ngữ mà không cần nhớ kiến thức:

1. Kỹ thuật "Tẩy trắng sự kiện" (Entity Masking / De-factualization) cho Data Bạn không cần tự viết ra kho dữ liệu ngôn ngữ tự nhiên mới, mà có thể lấy chính dữ liệu văn bản chất lượng cao có sẵn, nhưng biến đổi nó trước khi cho model học.

- Thay vì cho model học câu gốc: *"Năm 1911, Rutherford phát hiện ra hạt nhân nguyên tử tại Manchester."*
- Hãy dùng một pipeline xử lý để biến nó thành dạng trừu tượng hoặc gắn thẻ truy xuất: *"Vào [TIME_1], [ENTITY_A] đã phát hiện ra [CONCEPT_X] tại [LOCATION_B] -> (Dữ liệu đính kèm trong Context: TIME_1=1911, ENTITY_A=Rutherford...)"*.

- Khi train trên dữ liệu kiểu này (hoặc dùng hàm Loss bỏ qua không phạt model khi nó đoán sai tên riêng/sự kiện, chỉ phạt nặng khi nó sai cấu trúc câu, sai liên từ logic như *vì, tuy nhiên, do đó*), mạng nơ-ron sẽ tự động hiểu rằng: **"À, việc nhớ tên ông nào hay năm nào không giúp giảm Loss, việc duy nhất giúp giảm Loss là học cách copy thông tin từ Context vào đúng cấu trúc ngữ pháp và suy luận mối quan hệ giữa chúng."** Lúc này, 100% dung lượng trọng số của model nhỏ sẽ được dồn vào việc học "hình thái của tư duy và ngôn ngữ".

2. Kiến trúc nối thẳng Bộ nhớ vào Không gian ẩn (Latent-Space Retrieval) Để một model nhỏ dùng thông tin external mượt như thể nó biết sẵn trong đầu, cách làm RAG truyền thống (tìm văn bản rồi nhét vào đầu câu prompt) là chưa đủ tối ưu vì nó làm dài context và tốn compute xử lý lại từ đầu.

- Bạn có thể nhìn sang kiến trúc kiểu **RETRO (Retrieval-Enhanced Transformer)** của DeepMind hoặc **Memory Layers** mới gần đây. Trong kiến trúc này, model cực nhỏ (chỉ vài trăm triệu tham số), nhưng ở các layer giữa, nó có một cổng Cross-Attention chọc thẳng vào một Vector Database khổng lồ nằm trên ổ cứng SSD/RAM.

- Quá trình tra cứu diễn ra hoàn toàn tự động bằng vector ngay trong lúc model đang suy nghĩ (forward pass), chứ không cần sinh ra câu lệnh tìm kiếm bằng chữ. Model chỉ giữ nhiệm vụ lắp ghép các vector kiến thức lấy từ ngoài vào thành câu chữ mượt mà.

3. Dùng Model lớn làm "Máy chưng cất ngôn ngữ" (Synthetic Linguistic Distillation) Vì bạn đặt câu hỏi lấy đâu ra data ngôn ngữ tự nhiên tốt: Nguồn data tốt nhất hiện nay cho một SLM không phải là văn bản cào trên mạng về (vốn rất rác, dài dòng và nhiễu), mà là dùng chính các mô hình open-weight lớn hiện có (như Qwen hoặc DeepSeek) làm công cụ tiền xử lý để tạo ra **"Sách giáo khoa ngôn ngữ & tư duy"** (giống triết lý của dòng model Phi của Microsoft, nhưng cực đoan hơn).

- Bạn có thể viết script yêu cầu một model lớn sinh ra hàng triệu mẫu hội thoại, tranh luận, phân tích logic, chuyển đổi văn phong, tóm tắt... nhưng xoay quanh một tập hợp từ vựng và khái niệm cơ bản (Basic Vocabulary - khoảng 20.000 từ thông dụng nhất).

- Một bộ dataset chỉ khoảng **10 tỷ đến 20 tỷ token** nhưng cực kỳ tinh khiết, đa dạng về cấu trúc câu và hoàn toàn xoay quanh việc "đọc hiểu ngữ cảnh được cung cấp sẵn rồi suy luận" là đủ để train một model ~500M tham số từ con số 0 chỉ trong vài ngày trên một cụm GPU nhỏ (hoặc thậm chí vài tuần trên máy cá nhân mạnh), nhưng khả năng ăn nói và lý luận trên văn bản sẽ ngang ngửa các model lớn hơn nó hàng chục lần.

Tóm lại, nút thắt lớn nhất của việc tách ngôn ngữ ra khỏi kiến thức là **hiện tượng "Học vẹt theo quán tính" của Gradient Descent** — nếu trong data có sự kiện lặp đi lặp lại, mạng nơ-ron sẽ tự động ghi nhớ vào trọng số thay vì chịu khó nhìn ra bộ nhớ ngoài.

Để trị dứt điểm tật này trong lúc train, theo bạn mình nên can thiệp ở bước **tiền xử lý dữ liệu** (tráo đổi liên tục tên các thực thể/sự kiện trong văn bản để model không thể học thuộc lòng), hay can thiệp thẳng vào **kiến trúc mạng/hàm Loss** để ép nó phải phụ thuộc hoàn toàn vào luồng thông tin truyền từ ngoài vào?

User prompt: cái này tôi không rõ bằng bạn rồi. bạn cho tôi hướng dẫn cách train 1 small model đi 1. dữ liệu đối thoại tiếng Việt có thể sinh bằng cách gọi API2. bộ từ điển k cần quá nhiều từ 3. nó phải là 1 bộ lõi reasoning dựa vào external data như kiểu cắm thẳng vào vector database như bạn nói

Response: Để biến ý tưởng "SLM làm lõi tư duy + Bộ nhớ ngoài cắm thẳng vào mạng" thành một dự án chạy được trên máy cá nhân hoặc cụm GPU nhỏ, chúng ta sẽ xây dựng một kiến trúc lai giữa **Llama-style Decoder** (cho nhẹ và nhanh) và **RETRO (Retrieval-Enhanced Transformer)** của DeepMind.

Thay vì nhét văn bản tìm được vào câu prompt (làm dài chuỗi token và tốn compute tính Self-Attention), chúng ta sẽ mở một "cổng sau" (Cross-Attention) ở các tầng giữa của mạng nơ-ron. Model vừa đọc câu hỏi, vừa thò tay trực tiếp vào Vector Database lấy các vector kiến thức ra để nhào nặn thành câu trả lời.

Dưới đây là bản thiết kế kỹ thuật chi tiết qua 4 giai đoạn để bạn có thể bắt tay vào code ngay.

Giai đoạn 1: Xây dựng Bộ từ điển tối giản (Compact Tokenizer)

Vì mục tiêu của bạn là chỉ dạy cho model **cấu trúc ngôn ngữ tiếng Việt và tư duy logic** chứ không bắt nó nhớ tên riêng hay thuật ngữ chuyên ngành hiếm gặp, bộ từ điển (Vocabulary) cần phải ép xuống mức cực nhỏ.

- **Tại sao phải nhỏ?** Trong một model lớn, tầng Embedding (chuyển từ thành vector) và LM Head (chuyển vector thành xác suất từ) với vocab 128.000 từ có thể ngốn tới 300M - 500M tham số. Nếu bạn rút vocab xuống còn **16.384 (16k) hoặc 24.000 từ**, bạn tiết kiệm được hàng trăm triệu tham số để dồn toàn bộ "chất xám" vào các tầng Attention xử lý logic.

- **Cách làm:**

    1. Dùng thư viện `tokenizers` của Hugging Face (viết bằng Rust, chạy cực nhanh) để train một bộ **Byte-Level BPE** thuần tiếng Việt + ký hiệu toán/code.

    2. Chỉ đưa vào tập train tokenizer những văn bản hội thoại đời thường, sách giáo khoa logic, văn bản lập luận. Những từ ghép thông dụng tiếng Việt (*"tuy nhiên"*, *"do đó"*, *"nguyên nhân"*) sẽ được gộp thành 1 token. Còn những tên riêng lạ hoắc (như *"Rutherford"* hay *"Bỉm Sơn"*) cứ để nó tách thành từng mảnh ký tự nhỏ (sub-word/bytes), vì đằng nào model cũng chỉ copy từ bộ nhớ ngoài sang chứ không cần nhớ trong đầu.

    3. **Thêm các Special Tokens điều khiển bộ nhớ:** Bắt buộc thêm các token đặc biệt vào từ điển như `<|mem_start|>`, `<|mem_end|>`, `<|cite|>`, `<|unknown_fact|>`.

Giai đoạn 2: Pipeline sinh dữ liệu "Tẩy trắng sự kiện" bằng API

Đây là bước quyết định xem model nhỏ của bạn có bị "học vẹt" hay không. Bạn không dùng API (như DeepSeek hay Qwen) để sinh ra các đoạn chat hỏi đáp kiến thức thông thường, mà phải bắt API sinh ra các cặp dữ liệu **(Dữ liệu thô từ DB) + (Quá trình suy luận & Đối thoại)**, kèm theo kỹ thuật **Hoán vị thực thể (Counterfactual/Entity Swapping)**.

Cấu trúc một mẫu dữ liệu (Training Sample) bạn yêu cầu API sinh ra:

```
{
  "user_query": "Tại sao chất X-99 lại bị cấm sử dụng trong ngành thực phẩm ở nước Zoran vào mùa đông?",
  "external_memory_chunks": [
    "Chunk 1: Chất X-99 là một hợp chất bảo quản có nhiệt độ đóng băng ở 12 độ C.",
    "Chunk 2: Khi đóng băng, cấu trúc phân tử của X-99 bị bẻ gãy và giải phóng độc tố thần kinh nhẹ.",
    "Chunk 3: Nhiệt độ trung bình mùa đông tại quốc gia Zoran luôn dao động từ -5 đến 5 độ C."
  ],
  "assistant_reasoning_and_reply": "<|thought|> Người dùng hỏi lý do X-99 bị cấm ở Zoran vào mùa đông. Tra cứu
}
```

Mẹo cốt lõi khi gọi API sinh data:

- **Bịa đặt hóa thực thể (Anti-Memorization):** Trong system prompt gọi API, hãy yêu cầu model lớn thay thế ngẫu nhiên tên người, địa danh, con số, định luật thật bằng các tên giả định hoặc tráo đổi liên tục giữa các mẫu (ví dụ: mẫu 1 nói nước sôi ở 100 độ, mẫu 2 cho vào ngữ cảnh một hành tinh khác nơi nước sôi ở 45 độ). Khi thông tin sự kiện liên tục thay đổi giữa các mẫu train, Gradient Descent sẽ **bất lực trong việc ghi nhớ sự kiện vào trọng số**, và nó buộc phải học cách nhìn vào trường `external_memory_chunks` để lấy thông tin!
- **Đa dạng hóa tác vụ ngôn ngữ:** Yêu cầu API sinh đủ các sắc thái hội thoại tiếng Việt: tranh luận phản biện, tóm tắt ý, viết lại câu cho lịch sự hơn, bắt lỗi logic của người dùng, hoặc từ chối trả lời khi `external_memory_chunks` không chứa đủ thông tin (dạy model tính trung thực, không bịa chuyện).
- **Quy mô:** Chỉ cần khoảng **500.000 đến 1 triệu mẫu hội thoại** chất lượng cao như vậy (tương đương khoảng 500M - 1B tokens) là quá đủ để train một SLM nói tiếng Việt mượt như người thật.

Giai đoạn 3: Kiến trúc mạng "Lõi suy luận cắm thẳng Vector DB"

Thay vì dùng kiến trúc Transformer truyền thống, bạn thiết kế một mô hình khoảng **150M đến 300M tham số** (ví dụ: 16 layers, hidden dimension $d = 768$ hoặc 1024) gồm 2 thành phần phối hợp với nhau:

1. **Bộ mã hóa bộ nhớ (Frozen Memory Encoder):** Bạn không cần tự train phần này. Hãy lấy luôn một model embedding tiếng Việt nhỏ và rất mạnh có sẵn (hoặc cắt lấy 4-6 layer đầu của một model có sẵn, đóng băng toàn bộ trọng số `requires_grad = False`). Nhiệm vụ duy nhất của nó là biến các câu trong Vector Database (các `external_memory_chunks`) thành các ma trận vector đặc trưng $E \in \mathbb{R}^{M \times d_{mem}}$. Toàn bộ kho dữ liệu ngoài đời của bạn sẽ được băm sẵn thành vector và lưu trên ổ cứng thông qua các thư viện tìm kiếm tốc độ cao như **FAISS** hoặc **USearch/Qdrant**.
2. **Lõi Suy luận (Core Reasoning Decoder) và Cổng Gated Cross-Attention:** Lõi của bạn là một mạng Causal Transformer (như kiến trúc LLaMA/Mistral với RoPE, SwiGLU, RMSNorm). Tuy nhiên, cứ cách mỗi 3 hoặc 4 layer thông thường, bạn chèn thêm một tầng **Gated Cross-Attention** nối trực tiếp với các vector lấy từ Vector DB ra.

   Công thức hoạt động tại tầng thứ l có gắn cổng bộ nhớ rất gọn:

   - Đầu tiên, trạng thái ẩn của câu đang nói $H^{(l)}$ đi qua lớp Self-Attention bình thường để hiểu ngữ pháp và mạch hội thoại:

     $$H' = \text{SelfAttn}(H^{(l)})$$

   - Tiếp theo, dùng chính H' làm **Query (Q)** để chọc sang ma trận vector kiến thức E (được chiếu thành **Key (K)** và **Value (V)**) lấy từ Vector DB:

     $$\text{MemOut} = \text{CrossAttn}(Q = H'W_Q, K = EW_K, V = EW_V)$$

   - Đưa qua một cổng điều tiết (Gate) có tham số học được α (khởi tạo bằng 0 lúc mới bắt đầu train để mạng ổn định) trước khi đi vào mạng FFN:

     $$H^{(l+1)} = \text{FFN}\left(H' + \tanh(\alpha) \cdot \text{MemOut}\right)$$

Cơ chế tự động tra cứu khi chạy thực tế (Inference): Khi người dùng gõ câu hỏi vào, hoặc khi model vừa sinh ra hết một cụm từ (ví dụ một chunk 16 hoặc 32 tokens), vector ẩn ở tầng giữa của đoạn vừa sinh sẽ được trích xuất ngay lập tức, bắn thẳng sang FAISS (chỉ mất khoảng 1-2 mili-giây trên RAM) để kéo về K đoạn vector gần nhất từ ổ cứng. Các vector này được nạp thẳng vào cổng `CrossAttn` cho các bước sinh token tiếp theo mà không hề làm tốn chiều dài Context Window của câu chat!

Giai đoạn 4: Chiến lược Huấn luyện 2 Pha & Kỹ thuật "Bịt mắt" (Loss Masking)

Để ép model học đúng ý đồ với lượng phần cứng ít ỏi, bạn chia quá trình train (bằng PyTorch hoặc framework viết bằng Rust như `burn` / `candle`) làm 2 pha:

Pha 1: Học Ngôn ngữ & Logic thuần túy (Tắt cổng Cross-Attention)

- Cho cổng $\alpha = 0$ (tạm thời ngắt kết nối Vector DB).
- Đưa các thông tin cần thiết trực tiếp vào đầu câu prompt và train model với bài toán Next-Token Prediction tiêu chuẩn trên tập dữ liệu đã sinh ở Giai đoạn 2.

- **Kỹ thuật quan trọng - Selective Loss Masking:** Khi tính hàm Cross-Entropy Loss, bạn có thể giảm trọng số phạt (loss weight) đối với các token là danh từ riêng/con số cụ thể, và **nhân đôi trọng số phạt** đối với các từ nối logic (*vì vậy, mặc dù, suy ra, nếu... thì*), các động từ chỉ quan hệ và các token nằm trong thẻ `<|thought|>`. Model sẽ hội tụ cực nhanh và học cách lập luận tiếng Việt rất sắc bén chỉ sau vài epoch.

Pha 2: Mở cổng Bộ nhớ ngoài (Bật Cross-Attention)

- Lúc này bạn xóa phần thông tin đính kèm ở đầu câu prompt đi (để model không đọc được bằng mắt thường nữa), mà chuyển toàn bộ các `external_memory_chunks` đó thành dạng vector E bơm vào đường **Cross-Attention**.
- Bạn có thể đóng băng (freeze) một phần các lớp Self-Attention đã học tốt ngôn ngữ ở Pha 1, và tập trung cập nhật trọng số cho các ma trận W_Q, W_K, W_V của cổng Cross-Attention cùng tham số cổng α.
- Model lúc này bị rơi vào thế "bắt buộc": câu trả lời vẫn đòi hỏi phải có thông tin của chất `X-99` và nước `Zoran`, nhưng trong câu hỏi của user không hề có. Con đường duy nhất để giảm hàm Loss xuống bằng 0 là nó phải tự điều chỉnh các đầu đọc của `Cross-Attention` để hút đúng thông tin từ ma trận vector E bên ngoài vào và lắp vào cấu trúc câu tiếng Việt nó đã thuộc lòng ở Pha 1!

Tóm tắt cấu hình phần cứng & Thời gian dự kiến

Nếu bạn thiết kế model theo thông số **"Sweet Spot"** cho cá nhân nghiên cứu:

- **Kích thước lõi:** ~150M đến 250M tham số ($d_{model} = 768$, $12 - 16$ layers, 12 heads, Vocab 16.384).
- **Vector DB Encoder:** Dùng sẵn một model embedding nhỏ (như dòng BGE-small hoặc tương đương, ~30M-100M tham số, chạy ở chế độ `fp16` không tốn VRAM lưu gradient).
- **Tập dữ liệu:** 1 tỷ tokens dữ liệu hội thoại/suy luận chưng cất qua API.
- **Phần cứng:** Chỉ cần 1 đến 2 card **RTX 3090 / 4090 (24GB VRAM)** là bạn có thể train từ con số 0 (from scratch) toàn bộ quy trình này trong vòng **3 đến 5 ngày** với FlashAttention-2 và `bfloat16`. Khi chạy thực tế (inference), con model 200M tham số này có thể chạy vèo vèo ngay trên CPU hoặc iGPU laptop bình thường với tốc độ cả trăm token/giây, trong khi ổ cứng SSD chứa hàng chục Gigabyte vector kiến thức cắm ngoài.

Sơ đồ tương tác dưới đây mô tả trực quan cách luồng dữ liệu đi qua các khối của kiến trúc RETRO / Gated Cross-Attention này qua từng bước để bạn dễ hình dung khi bắt tay vào dựng các tầng mạng:

Bạn định dựng thử bộ khung kiến trúc mạng này trên PyTorch để tận dụng hệ sinh thái có sẵn trước, hay muốn viết thẳng bằng các framework trên Rust để kiểm soát bộ nhớ và luồng truy xuất từ đầu?
